

1、Redis和HashMap在结构上有什么区别？

Redis的特点：

- 支持分布式部署，适合高并发高可用场景
- 支持数据持久化，避免数据丢失
- 支持多种数据结构，功能丰富

HashMap的特点：

- 单机内存存储，不支持分布式
- 数据结构简单，功能单一
- 性能高，但受限于单机内存大小

2、Redis有哪些数据类型？底层是如何实现的？

Redis 5种基本数据类型： String、List、Hash、Set、Zset

3种特殊数据类型：

- **Bitmaps:** 位图，用于布隆过滤器、统计
- **HyperLogLog:** 基数统计
- **Geospatial:** 地理位置

以下笔记来自Redis体系课，面试突击课只是抽取其中一些快速讲解：

Redis提供了一些数据结构供我们往Redis中存取数据，最常用的有5种，字符串（String）、哈希（Hash）、列表（list）、集合（set）、有序集合（ZSET）。

字符串（String）

字符串类型是Redis最基础的数据结构。首先键都是字符串类型，而且其他几种数据结构都是在字符串类型基础上构建的，所以字符串类型能为其他四种数据结构的学习奠定基础。字符串类型的值实际可以是字符串(简单的字符串、复杂的字符串(例如JSON、XML))、数字(整数、浮点数)，甚至是二进制(图片、音频、视频)，但是值最大不能超过512MB。

（虽然Redis是C写的，C里面有字符串<本质使用char数组来实现>，但是出于种种考虑，Redis还是自己实现了字符串类型）

操作命令

set 设置值

set key value

```
} "lijin"  
27.0.0.1:6379> set key value [EX seconds|PX milliseconds|EXAT timestamp|PXAT milliseconds-timestamp|KEEPTTL] [NX|XX] [GET]
```

set命令有几个选项：

ex seconds: 为键设置秒级过期时间。

px milliseconds: 为键设置毫秒级过期时间。

nx: 键必须不存在,才可以设置成功，用于添加（分布式锁常用）。

xx: 与nx相反,键必须存在, 才可以设置成功,用于更新。

```
127.0.0.1:6880> set ex-hello redis ex 10
OK
127.0.0.1:6880> ttl ex-hello
(integer) 5
127.0.0.1:6880> ttl ex-hello
(integer) -2
```

从执行效果上看, ex参数和expire命令基本一样。还有一个需要特别注意的地方是如果一个字符串已经设置了过期时间, 然后你调用了set 方法修改了它, 它的过期时间会消失。

而nx和xx执行效果如下

```
127.0.0.1:6880> set ex-hello redis nx
OK
127.0.0.1:6880> set ex-hello redis nx
(nil)
127.0.0.1:6880> set ex-hello redis xx
OK
127.0.0.1:6880> set ex-hello redis2 xx
OK
127.0.0.1:6880> get ex-hello
"redis2"
```

除了set选项, Redis 还提供了setex和 setnx两个命令:

setex key
seconds value

setnx key value

setex和 setnx的作用和ex和nx选项是一样的。也就是, setex为键设置秒级过期时间, setnx设置时键必须不存在,才可以设置成功。

setex示例:

```
127.0.0.1:6880> setex ex-hello2 10 redis
OK
127.0.0.1:6880> ttl ex-hello2
(integer) 5
```

setnx示例:

```
127.0.0.1:6880> keys *
1) "hello"
2) "ex-hello"
3) "foo-ex"
4) "mark"
127.0.0.1:6880> setnx foo-ex redis
(integer) 0
127.0.0.1:6880> setnx foo-ex2 redis
(integer) 1
127.0.0.1:6880> keys *
1) "hello"
2) "foo-ex2"
3) "ex-hello"
4) "foo-ex"
5) "mark"
```

因为键foo-ex已存在,所以setnx失败,返回结果为0, 键foo-ex2不存在, 所以setnx成功,返回结果为1。

有什么应用场景吗?以setnx命令为例子, 由于Redis的单线程命令处理机制, 如果有多个客户端同时执行setnx key value, 根据setnx的特性只有一个客户端能设置成功, setnx可以作为分布式锁的一种实现方案。当然分布式锁没有不是只有一个命令就OK了, 其中还有很多的东西要注意, 我们后面会用单独的章节来讲述基于Redis的分布式锁。

get 获取值

如果要获取的键不存在,则返回nil(空):

```
127.0.0.1:6880> get ex-hello
"redis2"
127.0.0.1:6880> get ex-hello2
(nil)
```

mset 批量设置值

通过mset命令一次性设置4个键值对

```
127.0.0.1:6880> mset a 1 b 2 c 3 d 4
OK
```

mget 批量获取值

```
127.0.0.1:6880> mget a a1 b c d d2
1) "1"
2) (nil)
3) "2"
4) "3"
5) "4"
6) (nil)
```

批量获取了键a、b、c、d的值:

如果有些键不存在,那么它的值为nil(空), 结果是按照传入键的顺序返回。

批量操作命令可以有效提高效率, 假如没有mget这样的命令, 要执行n次get命令具体耗时如下:

$n \text{ 次 get 时间} = n \text{ 次网络时间} + n \text{ 次命令时间}$

使用mget命令后, 要执行n次get命令操作具体耗时如下:

$n \text{ 次 get 时间} = 1 \text{ 次网络时间} + n \text{ 次命令时间}$

Redis可以支撑每秒数万的读写操作, 但是这指的是Redis服务端的处理能力, 对于客户端来说, 一次命令除了命令时间还是有网络时间, 假设网络时间为1毫秒, 命令时间为0.1毫秒(按照每秒处理1万条命令算), 那么执行1000次 get命令需要1.1秒($1000 \times 1 + 1000 \times 0.1 = 1100\text{ms}$), 1次mget命令的需要0.101秒($1 \times 1 + 1000 \times 0.1 = 101\text{ms}$)。

Incr 数字运算

incr命令用于对值做自增操作,返回结果分为三种情况:

值不是整数,返回错误。

值是整数, 返回自增后的结果。

键不存在, 按照值为0自增,返回结果为1。

```
127.0.0.1:6880> exists incr-key
(integer) 0
127.0.0.1:6880> incr incr-key
(integer) 1
127.0.0.1:6880> incr incr-key
(integer) 2
127.0.0.1:6880> incr hello
(error) ERR value is not an integer or out of range
```

除了incr命令, Redis提供了decr(自减)、incrby(自增指定数字)、decrby(自减指定数字)、incrbyfloat(自增浮点数), 具体效果请同学们自行尝试。

append追加指令

append可以向字符串尾部追加值

```
127.0.0.1:6880> get hello
"redis"
127.0.0.1:6880> append hello study
(integer) 10
127.0.0.1:6880> get hello
"redisstudy"
```

strlen 字符串长度

返回字符串长度

```
127.0.0.1:6880> append hello " test"
(integer) 15
127.0.0.1:6880> get hello
"redisstudy test"
127.0.0.1:6880> strlen hello
(integer) 15
127.0.0.1:6880> set hello2 "中文"
OK
127.0.0.1:6880> get hello2
"\xe4\xb8\xad\xe6\x96\x87"
127.0.0.1:6880> strlen hello2
(integer) 6
```

注意：每个中文占3个字节

getset 设置并返回原值

getset和set一样会设置值,但是不同的是，它同时会返回键原来的值

```
127.0.0.1:6880> getset hello3 hello3
(nil)
127.0.0.1:6880> getset hello2 hello3
"\xe4\xb8\xad\xe6\x96\x87"
127.0.0.1:6880> getset hello hello3
"redisstudy test"
```

setrange 设置指定位置的字符

```
127.0.0.1:6880> get hello
"hello3"
127.0.0.1:6880> setrange hello 5 x
(integer) 6
127.0.0.1:6880> get hello
"hellox"
```

下标从0开始计算。

getrange 截取字符串

getrange 截取字符串中的一部分，形成一个子串，需要指明开始和结束的偏移量，截取的范围是个闭区间。

```
127.0.0.1:6880> get hello
"hello3"
127.0.0.1:6880> setrange hello 5 x
(integer) 6
127.0.0.1:6880> get hello
"hellox"
```

命令的时间复杂度

字符串这些命令中，除了del、mset、mget支持多个键的批量操作，时间复杂度和键的个数相关，为 $O(n)$ ，getrange和字符串长度相关，也是 $O(n)$ ，其余的命令基本上都是 $O(1)$ 的时间复杂度，在速度上还是非常快的。

使用场景

字符串类型的使用场景很广泛：

缓存功能

Redis 作为缓存层，MySQL作为存储层，绝大部分请求的数据都是从Redis中获取。由于Redis具有支撑高并发的特性,所以缓存通常能起到加速读写和降低后端压力的作用。

计数

使用Redis 作为计数的基础工具，它可以实现快速计数、查询缓存的功能,同时数据可以异步落地到其他数据源。

共享Session

一个分布式Web服务将用户的Session信息（例如用户登录信息）保存在各自服务器中，这样会造成一个问题，出于负载均衡的考虑，分布式服务会将用户的访问均衡到不同服务器上，用户刷新一次访问可能会发现需要重新登录，这个问题是用户无法容忍的。

为了解决这个问题,可以使用Redis将用户的Session进行集中管理,,在这种模式下只要保证Redis是高可用和扩展性的,每次用户更新或者查询登录信息都直接从Redis中集中获取。

限速

比如,很多应用出于安全的考虑,会在每次进行登录时,让用户输入手机验证码,从而确定是否是用户本人。但是为了短信接口不被频繁访问,会限制用户每分钟获取验证码的频率,例如一分钟不能超过5次。一些网站限制一个IP地址不能在一秒钟之内访问超过n次也可以采用类似的思路。

哈希(Hash)

Java里提供了HashMap, Redis中也有类似的数据结构,就是哈希类型。但是要注意,哈希类型中的映射关系叫作field-value,注意这里的value是指field对应的值,不是键对应的值。

操作命令

基本上,哈希的操作命令和字符串的操作命令很类似,很多命令在字符串类型的命令前面加上了h字母,代表是操作哈希类型,同时还要指明要操作的field的值。

hset设置

hset user:1 name lijn

```
127.0.0.1:6379> hset user:1 name lijn
(integer) 1
```

如果设置成功会返回1,反之会返回0。此外Redis提供了hsetnx命令,它们的关系就像set和setnx命令一样,只不过作用域由键变为field。

hget取值

hget user:1 name

如果键或field不存在,会返回nil。

```
127.0.0.1:6379> hset user:1 name lijn
(integer) 1
127.0.0.1:6379> hget user:1 name
"lijn"
127.0.0.1:6379> hget user:1 name1
(nil)
127.0.0.1:6379> hget user:2 name1
(nil)
```

hdel删除field

hdel会删除一个或多个field,返回结果为成功删除field的个数。

```
127.0.0.1:6880> hset user:1 age 18
(integer) 1
127.0.0.1:6880> hdel user:1 name agex
(integer) 1
127.0.0.1:6880> hdel user:1 agey
(integer) 0
```

hlen计算field个数

```
127.0.0.1:6379> hget user:1 age
"18"
127.0.0.1:6379> hlen user:1
(integer) 1
127.0.0.1:6379> hset user:1 name lijn
(integer) 1
127.0.0.1:6379> hlen user:1
(integer) 2
127.0.0.1:6379>
```

hmset批量设值

```
127.0.0.1:6379> hmset user:1 name lijn age 18 city beijing name msb age 20
OK
127.0.0.1:6379> hget user:1
(error) ERR wrong number of arguments for 'hget' command
127.0.0.1:6379> hget user:1 name
"msb"
```

hmget批量取值

```
127.0.0.1:6379> hmget user:1 name age city
1) "msb"
2) "20"
3) "beijing"
```

hexists判断field是否存在

```
(integer) 0
127.0.0.1:6379> hexists user:1 city
(integer) 1
127.0.0.1:6379> hexists user:1 city2
(integer) 0
127.0.0.1:6379>
```

若存在返回1，不存在返回0

hkeys获取所有field

它返回指定哈希键所有的field


```
(integer) 0
127.0.0.1:6379> hkeys user:1
1) "age"
2) "name"
3) "city"
```

hvals获取所有value

```
127.0.0.1:6379> hvals user:1
1) "20"
2) "msb"
3) "beijing"
```

hgetall获取所有field与value

```
127.0.0.1:6379> hgetall user:1
1) "age"
2) "20"
3) "name"
4) "msb"
5) "city"
6) "beijing"
```

在使用hgetall时，如果哈希元素个数比较多，会存在阻塞Redis的可能。如果只需要获取部分field，可以使用hmget，如果一定要获取全部field-value，可以使用hscan命令，该命令会渐进式遍历哈希类型，hscan将在后面的章节介绍。

hincrby增加

hincrby和 hincrbyfloat，就像incrby和incrbyfloat命令一样，但是它们的作用域是field。

hstrlen 计算value的字符串长度

```
127.0.0.1:6880> hstrlen user:1 name
(integer) 4
127.0.0.1:6880> hstrlen user:1 age
(integer) 2
```

命令的时间复杂度

哈希类型的操作命令中，hdel,hmget,hmset的时间复杂度和命令所带的field的个数相关 $O(k)$ ，hkeys,hgetall,hvals和存储的field的总数相关， $O(N)$ 。其余的命令时间复杂度都是 $O(1)$ 。

使用场景

从前面的操作可以看出，String和Hash的操作非常类似，那为什么要弄一个hash出来存储。

哈希类型比较适宜存放对象类型的数据，我们可以比较下，如果数据库中表记录user为：

id	name	age
1	lijin	18
2	msb	20

1、使用String类型

需要一条条去插入获取。

```
set user:1:name lijin;
```

```
set user:1:age 18;
```

```
set user:2:name msb;
```

```
set user:2:age 20;
```

优点：简单直观，每个键对应一个值

缺点：键数过多，占用内存多，用户信息过于分散，不用于生产环境

2、将对象序列化存入redis

```
set user:1 serialize(userInfo);
```

优点：编程简单，若使用序列化合理内存使用率高

缺点：序列化与反序列化有一定开销，更新属性时需要把userInfo全取出来进行反序列化，更新后再序列化到redis

3、使用hash类型

```
hmset user:1 name lijin age 18
```

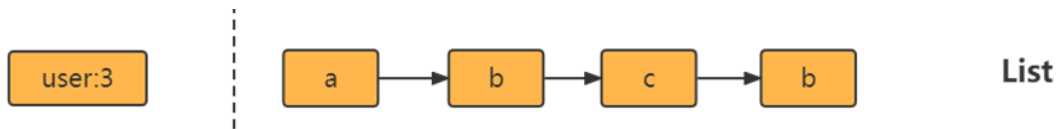
```
hmset user:2 name msb age 20
```

优点：简单直观，使用合理可减少内存空间消耗

缺点：要控制内部编码格式，不恰当的格式会消耗更多内存

列表 (list)

列表(list)类型是用来存储多个有序的字符串，a、b、c、c、b四个元素从左到右组成了一个有序的列表，列表中的每个字符串称为元素(element)，一个列表最多可以存储($2^{32}-1$)个元素(4294967295)。



在Redis 中，可以对列表两端插入(push)和弹出(pop)，还可以获取指定范围的元素列表、获取指定索引下标的元素等。列表是一种比较灵活的数据结构，它可以充当栈和队列的角色，在实际开发上有很多应用场景。

列表类型有两个特点：

第一、列表中的元素是有序的，这就意味着可以通过索引下标获取某个元素或者某个范围内的元素列表。

第二、列表中的元素可以是重复的。

操作命令

lrange 获取指定范围内的元素列表（不会删除元素）

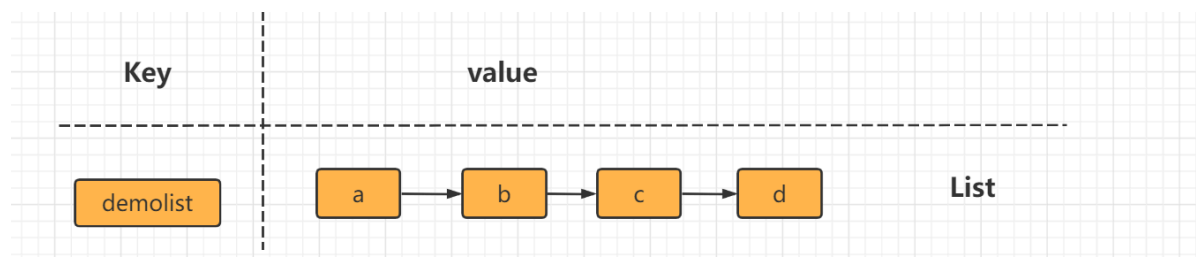
key start end

索引下标特点：从左到右为0到N-1

lrange 0 -1命令可以从左到右获取列表的所有元素

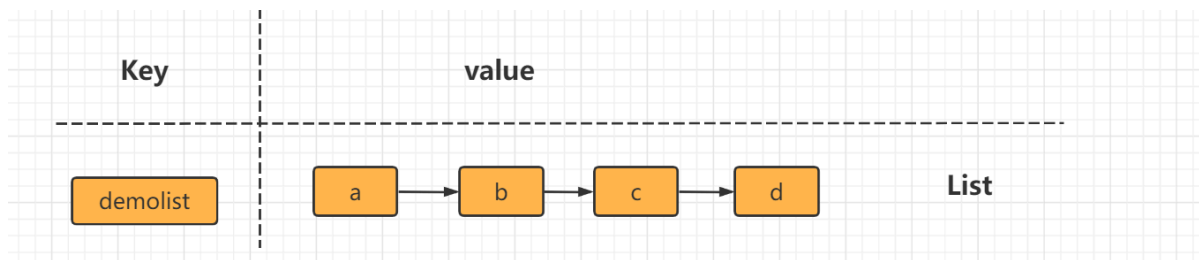
rpush 向右插入

```
127.0.0.1:6379> rpush demolist a
(integer) 1
127.0.0.1:6379> rpush demolist b
(integer) 2
127.0.0.1:6379> rpush demolist c
(integer) 3
127.0.0.1:6379> rpush demolist d
(integer) 4
127.0.0.1:6379> lrange demolist 0 -1
1) "a"
2) "b"
3) "c"
4) "d"
```



lpush 向左插入

```
127.0.0.1:6379> lpush demolist1 d
(integer) 1
127.0.0.1:6379> lpush demolist1 c
(integer) 2
127.0.0.1:6379> lpush demolist1 b
(integer) 3
127.0.0.1:6379> lpush demolist1 a
(integer) 4
127.0.0.1:6379> lrange demolist1 0 -1
1) "a"
2) "b"
3) "c"
4) "d"
```



linsert 在某个元素前或后插入新元素

```
127.0.0.1:6880> rpush enjoylist a
(integer) 1
127.0.0.1:6880> lpush enjoylist b
(integer) 2
127.0.0.1:6880> lrange enjoylist 0 -1
1) "b"
2) "a"
```

```
127.0.0.1:6880> rpush enjoylist a
(integer) 1
127.0.0.1:6880> lpush enjoylist b
(integer) 2
127.0.0.1:6880> lrange enjoylist 0 -1
1) "b"
2) "a"
```

```
127.0.0.1:6880> lrange enjoylist 0 -1
1) "b"
2) "java"
3) "android"
4) "a"
```

这三个返回结果为命令完成后当前列表的长度，也就是列表中包含的元素个数，同时rpush和lpush都支持同时插入多个元素。

lpop 从列表左侧弹出（会删除元素）

```
127.0.0.1:6379> lpop demolist  
"a"  
127.0.0.1:6379> lpop demolist  
"b"  
127.0.0.1:6379> lpop demolist  
"c"  
127.0.0.1:6379> lpop demolist  
"d"  
127.0.0.1:6379> lpop demolist  
(nil)
```

请注意，弹出来元素就没了。

rpop 从列表右侧弹出

rpop将会把列表最右侧的元素d弹出。

```
(nil)  
127.0.0.1:6379> rpop demolist1  
"d"  
127.0.0.1:6379> rpop demolist1  
"c"  
127.0.0.1:6379> rpop demolist1  
"b"  
127.0.0.1:6379> rpop demolist1  
"a"  
127.0.0.1:6379> rpop demolist1  
(nil)
```

lrem 对指定元素进行删除

```
) "x"  
127.0.0.1:6379> lrem key count element
```

lrem命令会从列表中找到等于value的元素进行删除，根据count的不同分为三种情况：

count>0, 从左到右,删除最多count个元素。

count<0, 从右到左,删除最多count绝对值个元素。

count=0, 删除所有。

```
(nil)  
127.0.0.1:6379> rpush demolist x x x x  
(integer) 4  
127.0.0.1:6379> lpush demolist y y y y  
(integer) 8
```

```
(01.01) Error: unknown command: lrange  
127.0.0.1:6379> lrange demolist 0 -1  
1) "y"  
2) "y"  
3) "y"  
4) "y"  
5) "x"  
6) "x"  
7) "x"  
8) "x"
```

```
127.0.0.1:6379> lrem demolist 2 y  
(integer) 2  
127.0.0.1:6379> lrange demolist 0 -1  
1) "y"  
2) "y"  
3) "x"  
4) "x"  
5) "x"  
6) "x"
```

```
127.0.0.1:6379> lrem demolist 1 x  
(integer) 1  
127.0.0.1:6379> lrange demolist 0 -1  
1) "y"  
2) "y"  
3) "x"  
4) "x"  
5) "x"
```

返回值是实际删除元素的个数。

ltrim 按照索引范围修剪列表

例如想保留列表中第0个到第1个元素

```
127.0.0.1:6379> ltrim demolist 0 1  
OK  
127.0.0.1:6379> lrange demolist 0 -1  
1) "y"  
2) "y"
```


lset修改指定索引下标的元素

```
127.0.0.1:6379> lset demolist 1 yyy
OK
127.0.0.1:6379> lrange demolist 0 -1
1) "y"
2) "yyy"
```

lindex 获取列表指定索引下标的元素

```
127.0.0.1:6379> lindex demolist 1
"yyy"
```

llen 获取列表长度

```
127.0.0.1:6379> llen demolist
(integer) 2
```

blpop和brpop阻塞式弹出元素

blpop和brpop是lpop和rpop的阻塞版本，除此之外还支持多个列表类型，也支持设定阻塞时间，单位秒，如果阻塞时间为0，表示一直阻塞下去。我们以brpop为例说明。

```
127.0.0.1:6880> brpop list:1 3
(nil)
(3.02s)
```

A客户端阻塞了（因为没有元素就会阻塞）

```
127.0.0.1:6379> blpop demolist 0
```

阻塞了

A客户端一直处于阻塞状态。此时我们从另一个客户端B执行

```
(integer) 1
127.0.0.1:6379> lpush demolist a
(integer) 1
127.0.0.1:6379>
```

b客户端

A客户端则输出

```
127.0.0.1:6379> blpop demolist 0
1) "demolist"
2) "a"
(59.91s)
```

拿到数据了

注意：brpop后面如果是多个键，那么brpop会从左至右遍历键，一旦有一个键能弹出元素，客户端立即返回。

使用场景

列表类型可以用于比如：

消息队列，Redis 的 `lpush+brpop` 命令组合即可实现阻塞队列，生产者客户端使用 `lpush` 从列表左侧插入元素，多个消费者客户端使用 `brpop` 命令阻塞式的“抢”列表尾部的元素，多个客户端保证了消费的负载均衡和高可用性。

文章列表

每个用户有属于自己的文章列表，现需要分页展示文章列表。此时可以考虑使用列表，因为列表不但是有序的，同时支持按照索引范围获取元素。

实现其他数据结构

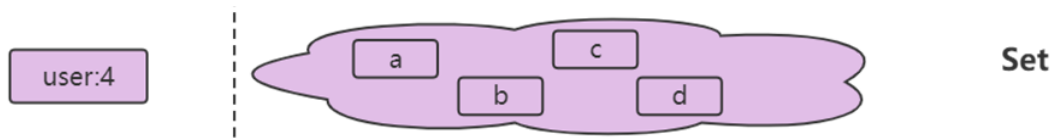
`lpush+lpop` = Stack (栈)

`lpush + rpop` = Queue (队列)

`lpush+ ltrim` = Capped Collection (有限集合)

`lpush+brpop` = Message Queue (消息队列)

集合 (set)



集合 (set) 类型也是用来保存多个的字符串元素，但和列表类型不一样的是，集合中不允许有重复元素，并且集合中的元素是无序的，不能通过索引下标获取元素。

一个集合最多可以存储 $2^{32}-1$ 个元素。Redis 除了支持集合内的增删改查，同时还支持多个集合取交集、并集、差集，合理地使用好集合类型，能在实际开发中解决很多实际问题。

集合内操作命令

sadd 添加元素

允许添加多个，返回结果为添加成功的元素个数

```
127.0.0.1:6379> sadd demoset a b c d a b c d
(integer) 4
```

srem 删除元素

允许删除多个，返回结果为成功删除元素个数

```
(integer) 4
127.0.0.1:6379> srem demoset c d
(integer) 2
```

scard 计算元素个数

```
(integer) 2
127.0.0.1:6379> scard demoset
(integer) 2
```

sismember 判断元素是否在集合中

如果给定元素element在集合内返回1，反之返回0

```
127.0.0.1:6379> sismember demoset a
(integer) 1
127.0.0.1:6379> sismember demoset d
(integer) 0
```

srandmember 随机从集合返回指定个数元素

指定个数如果不写默认为1

```
127.0.0.1:6379> srandmember demoset
"a"
127.0.0.1:6379> srandmember demoset 2
1) "b"
2) "a"
127.0.0.1:6379> srandmember demoset 3
1) "b"
2) "a"
```

spop 从集合随机弹出元素

同样可以指定个数，如果不写默认为1，注意，既然是弹出，spop命令执行后,元素会从集合中删除,而srandmember不会。

```
127.0.0.1:6379> spop demoset 1
1) "a"
```

smembers 获取所有元素(不会弹出元素)

返回结果是无序的

```
127.0.0.1:6379> smembers demoset
1) "b"
127.0.0.1:6379> smembers demoset
1) "b"
```

集合间操作命令

现在有两个集合,它们分别是set1和set2

```
127.0.0.1:6379> sadd set1 a b c d e f
(integer) 6
127.0.0.1:6379> sadd set2 d e f g h i
(integer) 6
```

sinter 求多个集合的交集

```
1) 0
127.0.0.1:6379> sadd set1 a b c d e f
(integer) 6
127.0.0.1:6379> sadd set2 d e f g h i
(integer) 6
127.0.0.1:6379> sinter set1 set2
1) "d"
2) "f"
3) "e"
```

sunion 求多个集合的并集

```
127.0.0.1:6379> sunion set1 set2
1) "i"
2) "b"
3) "f"
4) "a"
5) "e"
6) "h"
7) "d"
8) "c"
9) "g"
```

sdiff 求多个集合的差集

```
127.0.0.1:6379> sdiff set1 set2
1) "c"
2) "b"
3) "a"
127.0.0.1:6379> sdiff set2 set1
1) "h"
2) "g"
3) "i"
```

将交集、并集、差集的结果保存

```
sinterstore destination key [key ...]
sunionstore destination key [key ...]
sdiffstore destination key [key ...]
```

集合间的运算在元素较多的情况下会比较耗时，所以 Redis 提供了上面三个命令(原命令+store)将集合间交集、并集、差集的结果保存在 destination key 中，例如：

```

127.0.0.1:6379> sdiff set2 set1
1) "h"
2) "g"
3) "i"
127.0.0.1:6379> sdiffstore setcha set2 set1
(integer) 3
127.0.0.1:6379> smembers setcha
1) "h"
2) "g"
3) "i"
127.0.0.1:6379>

```

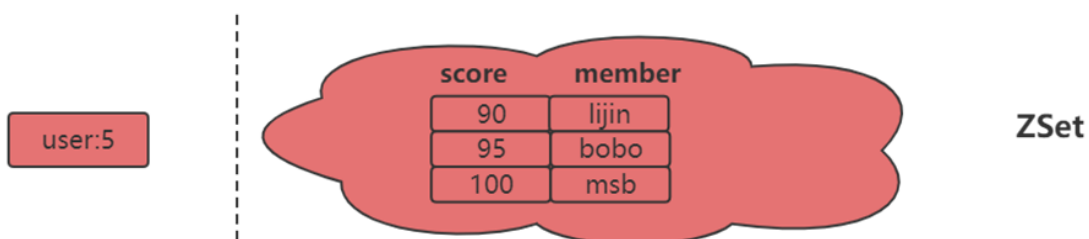
使用场景

集合类型比较典型的使用场景是标签(tag)。例如一个用户可能对娱乐、体育比较感兴趣，另一个用户可能对历史、新闻比较感兴趣，这些兴趣点就是标签。有了这些数据就可以得到喜欢同一个标签的人，以及用户的共同喜好的标签，这些数据对于用户体验以及增强用户黏度比较重要。

例如一个电子商务的网站会对不同标签的用户做不同类型的推荐，比如对数码产品比较感兴趣的人，在各个页面或者通过邮件的形式给他们推荐最新的数码产品，通常会为网站带来更多的利益。

除此之外，集合还可以通过生成随机数进行比如抽奖活动，以及社交图谱等等。

有序集合 (ZSET)



有序集合相对于哈希、列表、集合来说会有一点点陌生,但既然叫有序集合,那么它和集合必然有着联系,它保留了集合不能有重复成员的特性,但不同的是,有序集合中的元素可以排序。但是它和列表使用索引下标作为排序依据不同的是,它给每个元素设置一个分数(score)作为排序的依据。

有序集合中的元素不能重复，但是score可以重复，就和一个班里的同学学号不能重复,但是考试成绩可以相同。

有序集合提供了获取指定分数和元素范围查询、计算成员排名等功能，合理的利用有序集合，能帮助我们在实际开发中解决很多问题。

集合内操作命令

zadd添加成员

```

127.0.0.1:6379> zadd zset 90 lijin
(integer) 1
127.0.0.1:6379> zadd zset 95 bobo
(integer) 1
127.0.0.1:6379> zadd zset 100 msb
(integer) 1

```

返回结果代表成功添加成员的个数

要注意:

```
127.0.0.1:6379> zadd key [NX|XX] [GT|LT] [CH] [INCR] score member [score member ...]
```

zadd命令还有四个选项nx、xx、ch、incr 四个选项

nx: member必须不存在, 才可以设置成功, 用于添加。

xx: member必须存在, 才可以设置成功, 用于更新。

ch: 返回此次操作后, 有序集合元素和分数发生变化的个数

incr: 对score做增加, 相当于后面介绍的zincrby

zcard 计算成员个数

```
127.0.0.1:6379> zcard zset  
(integer) 3
```

zscore 计算某个成员的分数

```
127.0.0.1:6379> zscore zset lijn  
"90"  
127.0.0.1:6379> zscore zset lijn1  
(nil)
```

如果成员不存在则返回nil

zrank计算成员的排名

```
127.0.0.1:6379> zrank zset lijn  
(integer) 0  
127.0.0.1:6379> zrank zset msb  
(integer) 2
```

zrank是从分数从低到高返回排名

zrevrank反之

很明显, 排名从0开始计算。

zrem 删除成员

```
127.0.0.1:6379> zrem zset lijn  
(integer) 1  
127.0.0.1:6379> zrank zset lijn  
(nil)
```

允许一次删除多个成员。

返回结果为成功删除的个数。

zincrby 增加成员的分数

```
127.0.0.1:6379> zadd zset 90 lijn
(integer) 1
127.0.0.1:6379> zincrby zset 6 lijn
"96"
127.0.0.1:6379> zscore zset lijn
"96"
```

zrange和zrevrange返回指定排名范围的成员

有序集合是按照分值排名的，zrange是从低到高返回,zrevrange反之。如果加上withscores选项，同时会返回成员的分数

```
127.0.0.1:6379> zrange zset 0 3
1) "bobo"
2) "lijn"
3) "msb"
```

```
127.0.0.1:6379> zrange zset 0 3 withscores
1) "bobo"
2) "95"
3) "lijn"
4) "96"
5) "msb"
6) "100"
```

zrangebyscore返回指定分数范围的成员

```
zrangebyscore key min max [withscores] [limit offset count]
zrevrangebyscore key max min [withscores] [limit offset count]
```

其中zrangebyscore按照分数从低到高返回，zrevrangebyscore反之。例如下面操作从低到高返回200到221分的成员，withscores选项会同时返回每个成员的分数。

同时min和max还支持开区间(小括号) 和闭区间(中括号)，-inf和+inf分别代表无限小和无限大:

```
127.0.0.1:6379> zrangebyscore zset -inf +inf
1) "bobo"
2) "lijn"
3) "msb"
127.0.0.1:6379> zrangebyscore zset -inf +inf withscores
1) "bobo"
2) "95"
3) "lijn"
4) "96"
5) "msb"
6) "100"
```



```
127.0.0.1:6379> zrangebyscore zset 96 100 withscores
1) "lijin"
2) "96"
3) "msb"
4) "100"
```

```
127.0.0.1:6379> zrevrangebyscore zset +inf -inf withscores
1) "msb"
2) "100"
3) "lijin"
4) "96"
5) "bobo"
6) "95"
```

注意递减 范围也应该一致

zcount 返回指定分数范围成员个数

zcount key min max

```
127.0.0.1:6379> zcount zset 96 100
(integer) 2
127.0.0.1:6379> zcount zset 90 100
(integer) 3
```

zremrangebyrank 按升序删除指定排名内的元素

zremrangebyrank key start end

zremrangebyscore 删除指定分数范围的成员

zremrangebyscore key min max

集合间操作命令

zinterstore 交集

zinterstore

```
127.0.0.1:6379> zinterstore destination numkeys key [key ...] [WEIGHTS weight] [AGGREGATE SUM|MIN|MAX]
```

这个命令参数较多，下面分别进行说明

destination:交集计算结果保存到键。

numkeys:需要做交集计算键的个数。

key [key ...]:需要做交集计算的键。

weights weight

[weight ...]:每个键的权重，在做交集计算时，每个键中的每个member 会将自己分数乘以这个权重,每个键的权重默认是1。

aggregate sum/

min | max:计算成员交集后，分值可以按照sum(和)、min(最小值)、max(最大值)做汇总,默认值是sum。

不太好理解，我们用一个例子来说明。（算平均分）

```

(error) ERR value is not a valid float
127.0.0.1:6379> zadd zset1 90 lijn 95 bobo 100 msb
(integer) 3
127.0.0.1:6379> zadd zset2 99 lijn 96 bobo 90 msb
(integer) 3

127.0.0.1:6379> zinterstore zsetavg 2 zset1 zset2 weights 0.5 0.5
(integer) 3
127.0.0.1:6379> zrange zsetavg 0 -1 withscores
1) "lijn"
2) "94.5"
3) "msb"
4) "95"
5) "bobo"
6) "95.5"

```

zunionstore 并集

该命令的所有参数和zinterstore是一致的，只不过是做并集计算，大家可以自行实验。

使用场景

有序集合比较典型的使用场景就是排行榜系统。例如视频网站需要对用户上传的视频做排行榜，榜单的维度可能是多个方面的:按照时间、按照播放数量、按照获得的赞数。

3、Redis的持久化机制有哪些？RDB和AOF有什么区别？

1、RDB (Redis Database)

RDB持久化是把当前进程数据生成快照保存到硬盘的过程。所谓内存快照，就是指内存中的数据在某一个时刻的状态记录。这就类似于照片，当你给朋友拍照时，一张照片就能把朋友一瞬间的形象完全记录下来。RDB 就是Redis DataBase 的缩写。

2、AOF (Append Only File)

AOF(append only file)持久化:以独立日志的方式记录每次写命令，重启时再重新执行AOF文件中的命令达到恢复数据的目的。AOF的主要作用是解决了数据持久化的实时性,目前已经是Redis持久化的主流方式

4、Redis的过期键删除策略有哪些？

Redis 所有的数据结构都可以设置过期时间，时间一到，就会自动删除。但是会不会因为同一时间太多的key 过期，以至于忙不过来。同时因为Redis 是单线程的，删除的时间也会占用线程的处理时间，如果删除的太过于繁忙，会不会导致线上读写指令出现卡顿。

过期的 key 集合

redis 会将每个设置了过期时间的

key 放入到一个独立的字典中，以后会定时遍历这个字典来删除到期的 key。除了定时遍历之外，它还会使用惰性策略来删除过期的 key，所谓惰性策略就是在客户端访问这个 key 的时候，redis 对 key 的过期时间进行检查，如果过期了就立即删除。定时删除是集中处理，惰性删除是零散处理

惰性删除

所谓惰性策略就是在客户端访问这个key的时候，redis对key的过期时间进行检查，如果过期了就立即删除，不会给你返回任何东西。

从库的过期策略

从库不会进行过期扫描，从库对过期的处理是被动的。主库在 key 到期时，会在 AOF 文件里增加一条 del 指令，同步到所有的从库，从库通过执行这条 del 指令来删除过期的 key。

因为指令同步是异步进行的，所以主库过期的

key 的 del 指令没有及时同步到从库的话，会出现主从数据的不一致，主库没有的数据在从库里还存在，比如上一节的集群环境分布式锁的算法漏洞就是因为这个同步延迟产生的。

5、Redis的内存淘汰策略有哪些？

当 Redis 内存超出物理内存限制时，内存的数据会开始和磁盘产生频繁的交换 (swap)。交换会让 Redis 的性能急剧下降，对于访问量比较频繁的 Redis 来说，这样龟速的存取效率基本上等于不可用。

maxmemory

在生产环境中我们是不允许 Redis 出现交换行为的，为了限制最大使用内存，Redis 提供了配置参数 maxmemory 来限制内存超出期望大小。

当实际内存超出 maxmemory 时，Redis 提供了几种可选策略(maxmemory-policy) 来让用户自己决定该如何腾出新的空间以继续提供读写服务。

```
127.0.0.1:6379> config set maxmemory-policy volatile-lfu
OK
127.0.0.1:6379> exit

127.0.0.1:6379> config get maxmemory-policy
1) "maxmemory-policy"
2) "noeviction"
```

Noeviction

□ noeviction 不会继续服务写请求

(DEL 请求可以继续服务)，读请求可以继续进行。这样可以保证不会丢失数据，但是会让线上的业务不能持续进行。这是默认的淘汰策略。

volatile-lru

□ volatile-lru 尝试淘汰设置了过期时间的

key，最老使用的 key 优先被淘汰。没有设置过期时间的 key 不会被淘汰，这样可以保证需要持久化的数据不会突然丢失。

volatile-ttl

volatile-ttl 跟上面一样，除了淘汰的策略不是 LRU，而是 key 的剩余寿命 ttl 的值，ttl 越小越优先被淘汰。

volatile-random

volatile-random 跟上面一样，不过淘汰的 key 是过期 key 集合中随机的 key。

allkeys-lru

allkeys-lru 区别于volatile-lru，这个策略要淘汰的 key 对象是全体的 key 集合，而不只是过期的 key 集合。这意味着没有设置过期时间的 key 也会被淘汰。

allkeys-random

allkeys-random跟上面一样，不过淘汰的策略是随机的 key。

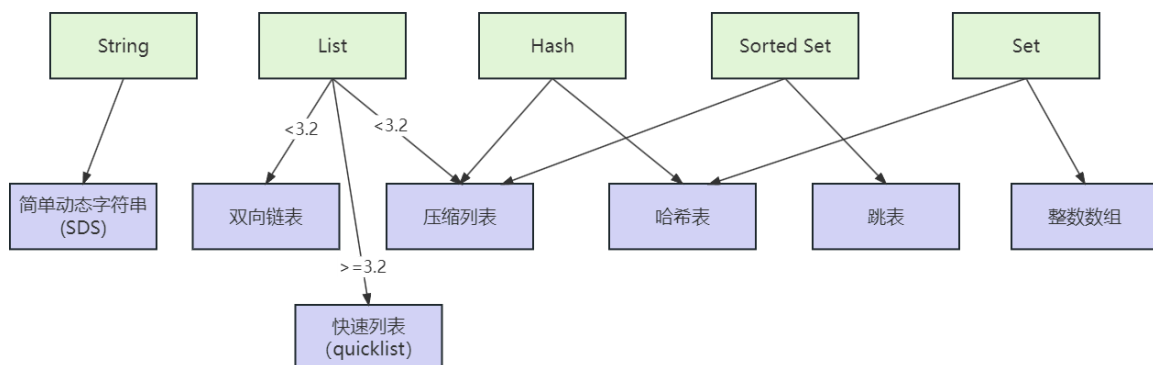
volatile-xxx 策略只会针对带过期时间的key 进行淘汰，allkeys-xxx 策略会对所有的 key 进行淘汰。如果你只是拿 Redis 做缓存，那应该使用 allkeys-xxx，客户端写缓存时不必携带过期时间。如果你还想同时使用 Redis 的持久化功能，那就使用 volatile-xxx 策略，这样可以保留没有设置过期时间的 key，它们是永久的 key 不会被 LRU 算法淘汰。

6、Redis的底层数据结构有哪些？

以下笔记来自Redis体系课，面试突击课只是抽取其中一些快速讲解：

五大数据类型底层实现

底层数据结构一共有 7 种，分别是简单动态字符串、双向链表、压缩列表、哈希表、跳表和整数数组、快速列表。它们和数据类型的对应关系如下图所示



简单动态字符串（Simple Dynamic String）

Redis 的String类型使用 SDS（简单动态字符串）（Simple Dynamic String）作为底层的数据结构实现。

SDS 与 C 字符串有所不同，它不仅可以保存文本数据，还可以保存二进制数据。这是因为 SDS 使用 len 属性的值而不是空字符来判断字符串是否结束，并且 SDS 的所有 API 都会以处理二进制的方式来处理 SDS 存放在 buf[] 数组里的数据。因此，SDS 不仅能存放文本数据，还能保存图片、音频、视频、压缩文件等二进制数据。

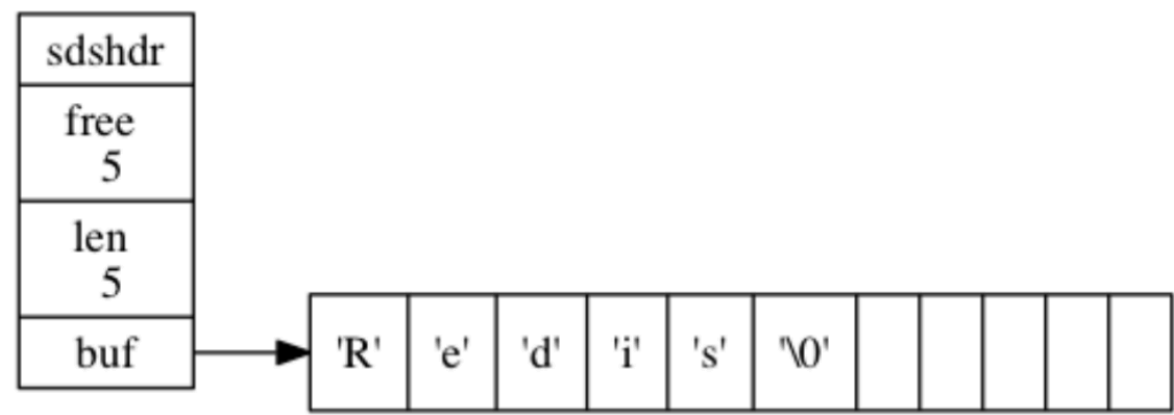
另外，Redis 的 SDS API 是安全的，拼接字符串不会造成缓冲区溢出。这是因为 SDS 在拼接字符串之前会检查 SDS 空间是否满足要求，如果空间不够会自动扩容，从而避免了缓冲区溢出的问题。

此外，获取字符串长度的时间复杂度是 $O(1)$ ，因为 SDS 结构里用 len 属性记录了字符串长度，所以获取长度的复杂度为 $O(1)$ 。相比之下，C 语言的字符串并不记录自身长度，所以获取长度的复杂度为 $O(n)$ 。这些特性使得 SDS 成为 Redis 的一个重要组成部分。

源码分析：

不同的版本的实现是有一些区别的。

老版本（3.2之前）



```
/*
 * 保存字符串对象的结构---这里不是真实的C的源码，我写的一个简化
 */
struct sdshdr {
    // buf 中已占用空间的长度
    int len;
    // buf 中剩余可用空间的长度
    int free;
    // 数据空间
    char buf[];
};
```

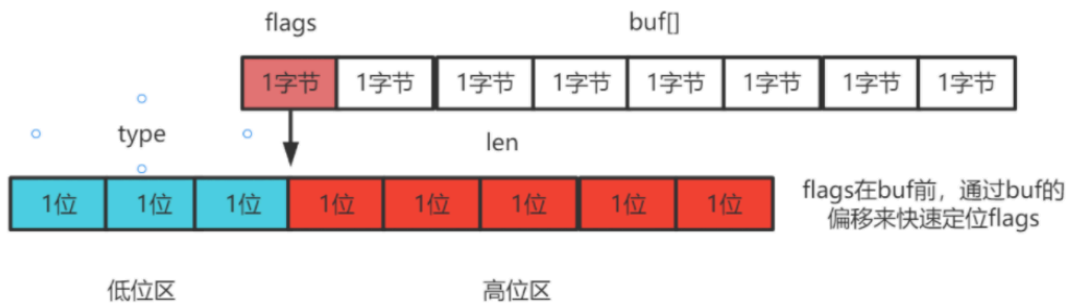
新版本（3.2之后）

在Redis的6及6以后，会根据字符串长度不同，定义了5种的SDS结构体sdshdr5、sdshdr8、sdshdr16、sdshdr32、sdshdr64，长度分别对应，2的n次幂（2的5次幂、2的8次幂.....），用于用于存储不同长度的字符串。

- 1. sdshdr5：适用于长度小于32<2的5次方>的字符串。
- 2. sdshdr8：适用于长度小于256<2的8次方>的字符串。
- 3. sdshdr16：适用于长度小于65535<2的16次方>的字符串。
- 4. sdshdr32：适用于长度小于4294967295<2的32次方>的字符串。
- 5. sdshdr64：适用于长度大于4294967295的字符串。

通过使用不同的sdshdr结构，Redis可以根据字符串的长度选择最合适的结构，从而提高内存利用率。例如，当我们存储一个长度为3字节的字符串时，Redis会选择使用sdshdr5结构，而不会浪费额外的内存空间。

sdshdr5结构如下：

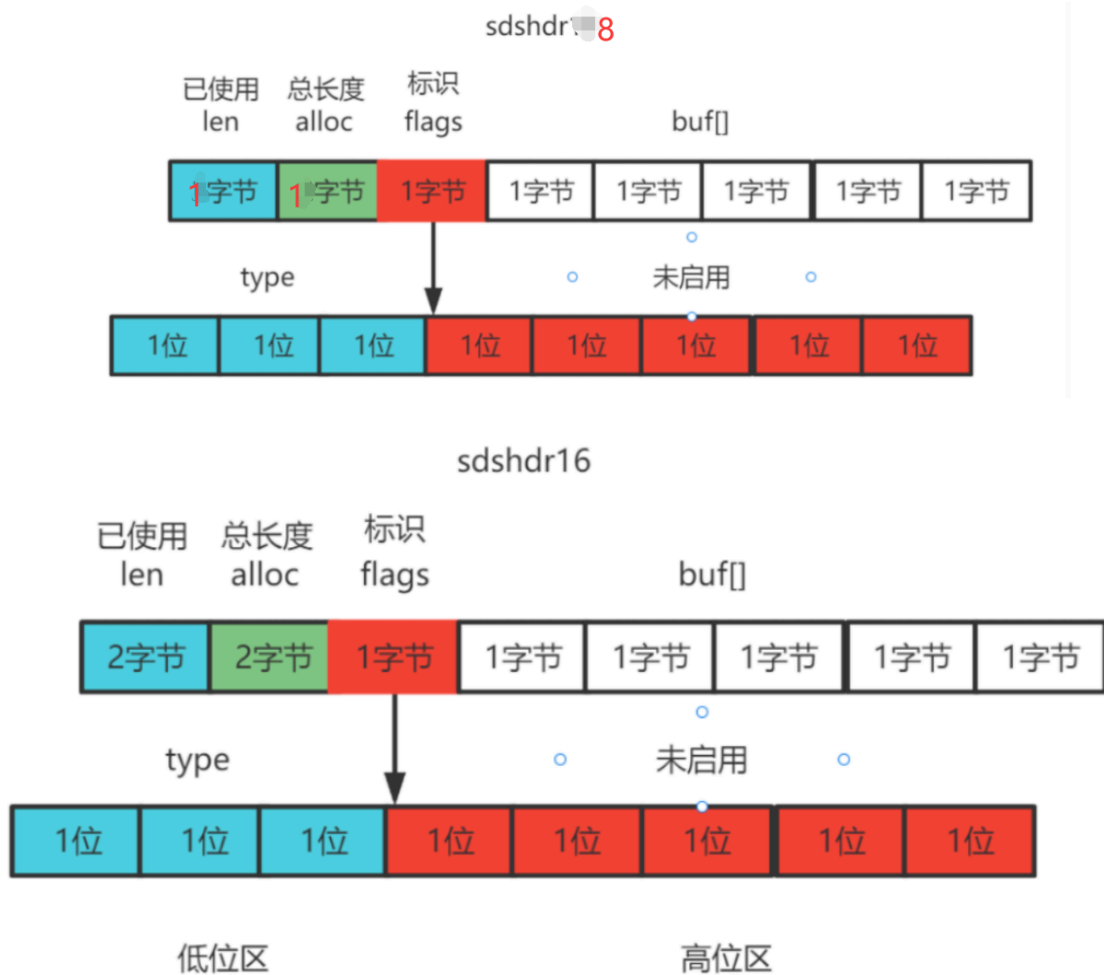


sdshdr5的结构中flags是一个char，其中低3位要标识type类型（就是存储的格式），所以就只有5位来存储len这个长度，所以就叫做sdshdr5

```
struct sdshdr5 {
    char flags;
    // 数据空间
    char buf[];
};
```

如果是更长的长度，Redis就需要采用sdshdr8或者sdshdr16或者更大的存储结构。

Redis的sdshdr5相对于sdshdr8少两个字段，是为了节省内存空间和提高处理短字符串的效率。根据字符串的长度范围选择适合的sdshdr结构，可以优化内存利用和性能。



free=alloc-len

sdshdr8

```
struct __attribute__((__packed__)) sdshdr8 {  
    uint8_t len; /* used */  
    uint8_t alloc; /* excluding the header and null terminator */  
    unsigned char flags; /* 3 lsb of type, 5 unused bits */  
    char buf[];  
};
```

sdshdr16

```
struct __attribute__((__packed__)) sdshdr16 {  
    uint16_t len; /* used */  
    uint16_t alloc; /* excluding the header and null terminator */  
    unsigned char flags; /* 3 lsb of type, 5 unused bits */  
    char buf[];  
};
```

sdshdr32

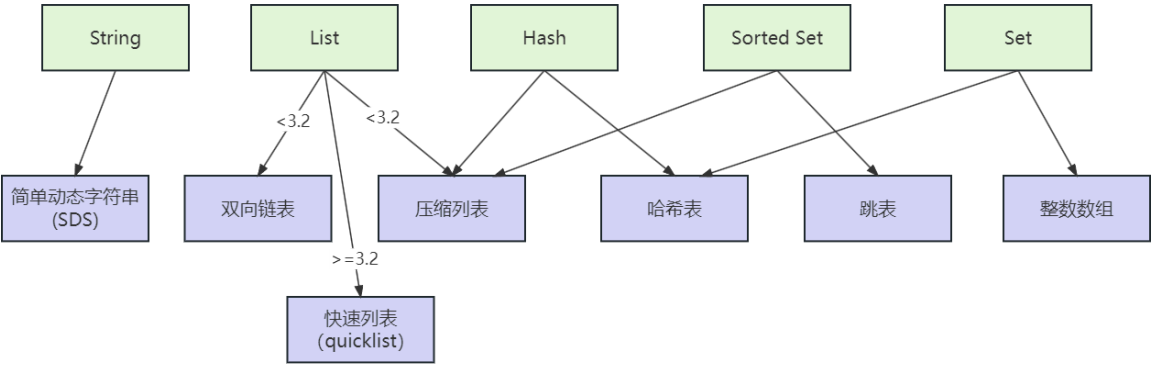
```
struct __attribute__((__packed__)) sdshdr32 {  
    uint32_t len; /* used */  
    uint32_t alloc; /* excluding the header and null terminator */  
    unsigned char flags; /* 3 lsb of type, 5 unused bits */  
    char buf[];  
};
```

sdshdr64

```
struct __attribute__((__packed__)) sdshdr64 {  
    uint64_t len; /* used */  
    uint64_t alloc; /* excluding the header and null terminator */  
    unsigned char flags; /* 3 lsb of type, 5 unused bits */  
    char buf[];  
};
```

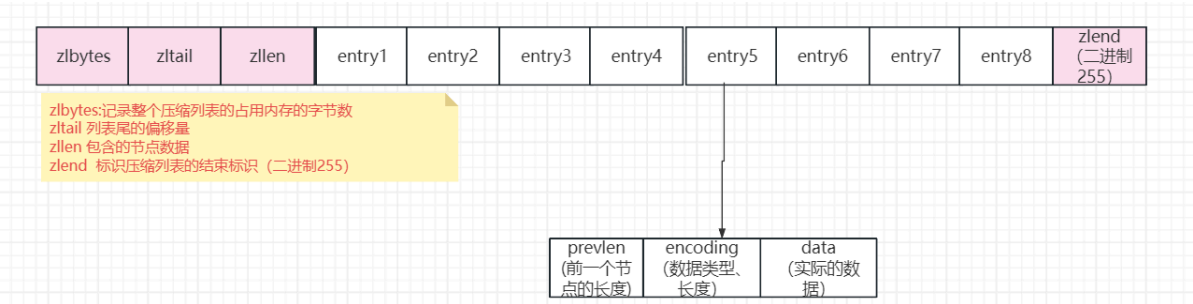
从String的设计上来看，Redis已经把空间利用做到了极致，同时的也可以从sdshdr5到sdshdr8...看出设计原则：开闭原则，对修改关闭，对拓展开放。

List



压缩列表

压缩列表实际上类似于一个数组，数组中的每一个元素都对应保存一个数据。和数组不同的是，压缩列表在表头有三个字段 `zlbytes`、`zltail` 和 `zllen`，分别表示列表长度、列表尾的偏移量和列表中的 `entry` 个数；压缩列表在表尾还有一个 `zlend`，表示列表结束。



在压缩列表中，如果我们要查找定位第一个元素和最后一个元素，可以通过表头三个字段的长度直接定位，复杂度是 $O(1)$ 。而查找其他元素时，就没有这么高效了，只能逐个查找，此时的复杂度就是 $O(N)$ 了。

- `zlbytes`，记录整个压缩列表占用对内存字节数
- `zltail`，记录压缩列表「尾部」节点距离起始地址由多少字节，也就是列表尾的偏移量
- `zllen`，记录压缩列表包含的节点数量；
- `zlend`，标记压缩列表的结束点，固定值 `0xFF`（十进制255）

压缩列表节点包含三部分内容：

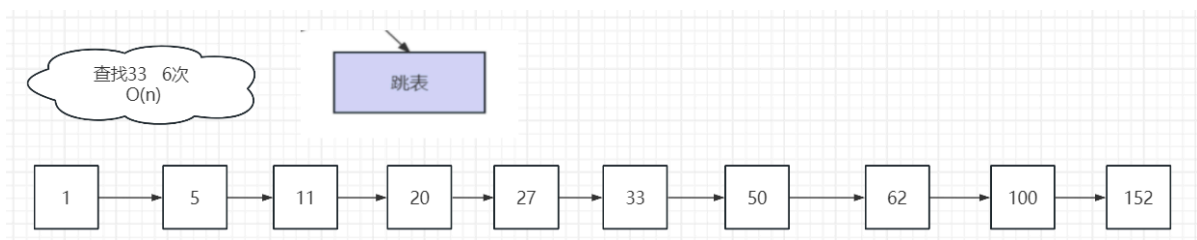
- `prevlen`，记录了「前一个节点」的长度
- `encoding`，记录了当前节点实际数据的类型以及长度
- `data`，记录了当前节点的实际数据

在压缩列表中，如果我们要查找定位第一个元素和最后一个元素，可以通过表头三个字段的长度直接定位，复杂度是 $O(1)$ 。而查找其他元素时，就没有这么高效了，只能逐个查找，此时的复杂度就是 $O(N)$ 了。因此压缩列表不适合保存过多的元素

跳表

跳表在链表的基础上，增加了多级索引，通过索引位置的几个跳转，实现数据的快速定位。

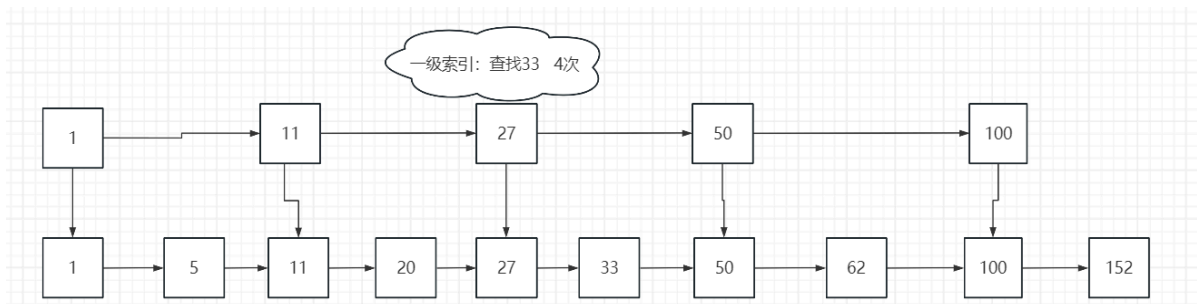
（链表只能逐一直找元素，导致操作起来非常缓慢，于是就出现了跳表）



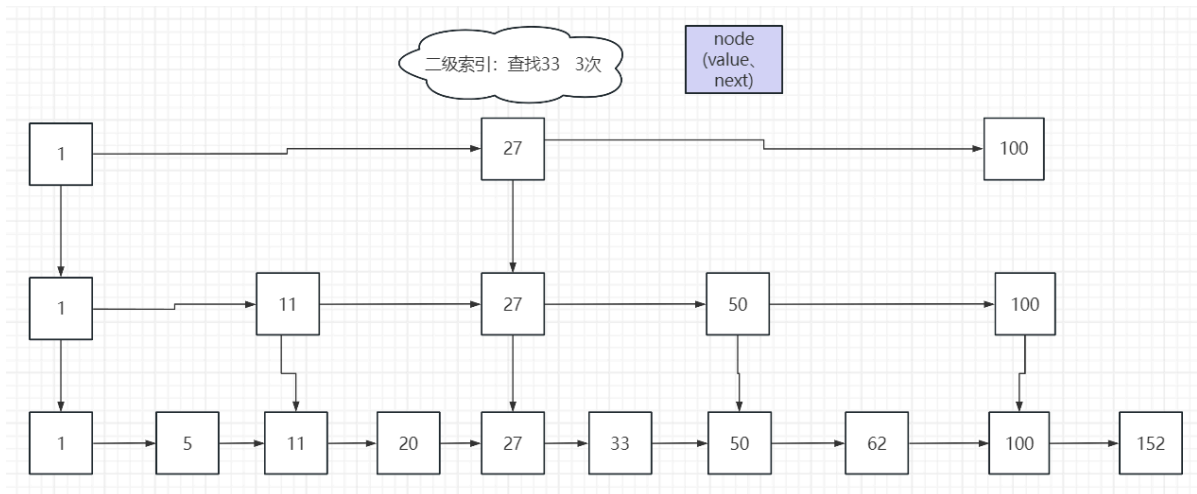
如果我们要在链表中查找 33 这个元素，只能从头开始遍历链表，查找 6 次，直到找到 33 为止。此时，复杂度是 $O(N)$ ，查找效率很低

1、为了提高查找速度，我们来增加一级索引：从第一个元素开始，每两个元素选一个出来作为索引。这些索引再通过指针指向原始的链表。

例如，从前两个元素中抽取元素 1 作为一级索引，从第三、四个元素中抽取元素 11 作为一级索引。此时，我们只需要 4 次查找就能定位到元素 33 了



2、我们还想再快，可以再增加二级索引：从一级索引中，再抽取部分元素作为二级索引。例如，从一级索引中抽取 1、27、100 作为二级索引，二级索引指向一级索引。这样，我们只需要 3 次查找，就能定位到元素 33 了。



这种本质就是空间换时间的算法。

使用选择

List

在 Redis 3.2 版本之前

Redis 的 List 类型底层数据结构可以由双向链表或压缩列表实现。如果列表元素个数小于 512 个且每个元素的值都小于 64 字节，则 Redis 会使用压缩列表作为底层数据结构；否则，Redis 会使用双向链表作为底层数据结构。

在 Redis 3.2 版本之后

List 类型底层数据结构只由 quicklist 实现，代替了双向链表和压缩列表。

String 类型的底层实现只有一种数据结构，也就是简单动态字符串。而 List、Hash、Set 和 Sorted Set 这四种集合类型，都有两种底层实现结构。

Hash

当一个Hash类型的键值对数量比较少时，Redis会使用压缩列表（ziplist）来表示Hash。当Hash类型的键值对数量较多时，会使用哈希表（hashtable）来表示Hash。哈希表在元素数量较多时具有更好的性能。

Sorted Set

当Sorted Set类型的成员数量较少（元素数量小于配置的压缩列表最大元素数量限制，默认为128）且成员的值较短时，Redis会使用压缩列表（ziplist）来表示Sorted Set。

当Sorted Set类型的成员数量较多或成员的值较长时，会使用跳表（skiplist）来表示Sorted Set。跳表在有序集合类型中提供了高效的范围查询操作。

Set

当Set类型的元素数量较少（元素数量小于配置的哈希最大压缩列表元素数量限制，默认为512）时，Redis会使用压缩列表（ziplist）来表示Set。当Set类型的元素数量较多时，会使用哈希表（hashtable）来表示Set。

7、Redis是单线程的吗？为什么这么快？

Redis确实是单线程！

主线程：处理命令执行

1. 内存操作：所有数据在内存中，O(1)操作
2. 单线程：无锁竞争，无上下文切换
3. 非阻塞IO：基于epoll的多路复用
4. 高效数据结构：底层数据结构优化
5. 避免了线程开销：无锁、无切换

8、Redis的事务能保证原子性吗？

Redis事务不完全保证原子性！

```
MULTI      # 开始事务
SET key value
GET key
EXEC        # 执行事务
DISCARD    # 取消事务
```

弱事务的特性。

Redis提供了简单的事务功能，将一组需要一起执行的命令放到multi和exec两个命令之间。multi 命令代表事务开始，exec命令代表事务结束。另外discard命令是回滚。

一个客户端

```

1) (integer) 1
127.0.0.1:6379> multi
OK
127.0.0.1:6379> sadd A-follow B
QUEUED
127.0.0.1:6379> sadd B-fans A
QUEUED
127.0.0.1:6379> exec
1) (integer) 1
2) (integer) 1
127.0.0.1:6379>

```

另外一个客户端

在事务没有提交的时查询（查不到数据）

```

127.0.0.1:6379> sismember A-follow B
(integer) 0
127.0.0.1:6379>

```

在事务提交后查询（可以查到数据）

```

127.0.0.1:6379> sismember A-follow B
(integer) 1
127.0.0.1:6379>

```

可以看到sadd命令此时的返回结果是QUEUED，代表命令并没有真正执行，而是暂时保存在Redis中的一个缓存队列（所以discard也只是丢弃这个缓存队列中的未执行命令，并不会回滚已经操作过的数据，这一点要和关系型数据库的Rollback操作区分开）。

只有当exec执行后，用户A关注用户B的行为才算完成，如下所示exec返回的两个结果对应sadd命令。

但是要注意Redis的事务功能很弱。在事务回滚机制上，Redis只能对基本的语法错误进行判断。

如果事务中的命令出现错误,Redis 的处理机制也不尽相同。

1、语法命令错误

```

127.0.0.1:6379> multi
OK
127.0.0.1:6379> sett lijn 1
(error) ERR unknown command `sett`, with args beginning with: `lijn`, `1`,
127.0.0.1:6379> set laoma 2
QUEUED
127.0.0.1:6379> exec
(error) EXECABORT Transaction discarded because of previous errors.
127.0.0.1:6379> get laoma
(nil)
127.0.0.1:6379>

```

例如下面操作错将set写成了sett，属于语法错误，会造成整个事务无法执行，事务内的操作都没有执行：

2、运行时错误

例如：事务内第一个命令简单的设置一个string类型，第二个对这个key进行sadd命令，这种就是运行时命令错误，因为语法是正确的：

```
127.0.0.1:6379> multi
OK
127.0.0.1:6379> set lijin 666
QUEUED
127.0.0.1:6379> sadd lijin 777
QUEUED
127.0.0.1:6379> exec
1) OK
2) (error) WRONGTYPE Operation against a key holding the wrong kind of value
127.0.0.1:6379> get lijin
"666"
127.0.0.1:6379>
```

可以看到Redis并不支持回滚功能，第一个set命令已经执行成功,开发人员需要自己修复这类问题。